

SPORTPHARM HQ — AZURE SETUP

For: whoever administers SportPharm's Microsoft 365 tenant and DNS. **Time:** about half a day, most of it waiting for admin consent and DNS. **Cost:** ~\$9/month (Azure Static Web Apps, Standard tier). Everything else is already paid for as part of Microsoft 365.

What this is

SportPharm HQ is an internal web app — orders to the fulfilment vendor, the team's week, marketing planning. It works today, but it is hosted as a public static site, so anyone with the link can read it and there is nowhere to keep a credential.

This puts it on Azure Static Web Apps behind Entra ID, so only SportPharm people and named partners can open it. The application code is already written, including the Azure Functions; this is provisioning, not development.

What this does NOT touch

sportpharm.com stays exactly as it is. The live WordPress/WooCommerce store is not moved, modified, migrated or taken offline at any point. No plugin, no theme change, no downtime, no DNS change to the apex.

HQ is added *alongside* it on a subdomain. The only contact between the two is that HQ later reads the store's order list through a **read-only** API key — and even that is Part 2 and can wait.

Two parts

Part 1 gets HQ running with real logins. That is all most people need, and it involves nobody outside your own admin access. HQ is fully usable at the end of it — the parts that need Part 2 explain what is missing rather than breaking.

Part 2 connects the store and mail when someone wants it. It needs a person with WordPress admin and a person with Stripe access, so it will move slower. Nothing in Part 1 has to wait for it.

Who does what

Split by who has access, so no secret has to pass through anyone who does not need it:

PART	WHO	WHY
Part 1 — app registration, guest access, DNS	You	Entra admin, HostPapa DNS
Part 1 — creating the Static Web App	Kennady	She owns the GitHub repo
Part 1 — the two auth settings	You	Secret should not pass through anyone else
Part 2 — Mail.Send and its restriction	You	Entra admin
Part 2 — WooCommerce key	Store admin	WordPress admin
Part 2 — Stripe key	Whoever has Stripe	Stripe dashboard

Do not send any secret back to Kennady or paste it into a chat. Put the values straight into Azure's Configuration blade. She only needs to know when each step is done.

PART 1 — HQ AND LOGIN

1 · App registration (Entra ID)

Entra ID → App registrations → New registration.

- Name: SportPharm HQ
- Supported account types: **this organizational directory only**
- Redirect URI: leave blank for now

Note the **Application (client) ID** and the **Directory (tenant) ID**. The tenant ID should be `c18f5ce2-49ee-4bb9-9c5e-9ddab1991d0c` — worth confirming it matches.

Certificates & secrets → New client secret. Copy the value now; it is not shown again.

API permissions are **not needed for Part 1** — sign-in does not require any. `Mail.Send` is added in Part 2, when HQ starts sending orders itself.

2 · Static Web App — *Kennady*

Azure → Create → Static Web App.

- **Plan: Standard.** Free does not support a custom Entra registration.
- Source: GitHub → `kennadyscott/sportpharm-content-review`, branch `main`
- Build presets: **Custom**
- App location: `/`
- Api location: `azure/api`
- Output location: leave blank

3 · Pin sign-in to this tenant

In the repo, `azure/staticwebapp.config.json` contains:

```
"openIdIssuer": "https://login.microsoftonline.com/<TENANT_ID>/v2.0"
```

Replace `<TENANT_ID>` with the real tenant ID. **This one value is what stops any Microsoft account in the world from signing in** — the default Entra provider is multi-tenant.

4 · Application settings

Static Web App → Configuration → Application settings:

For Part 1, two settings:

NAME	VALUE
<code>AAD_CLIENT_ID</code>	Application (client) ID from step 1
<code>AAD_CLIENT_SECRET</code>	The client secret from step 1

Part 2 adds the rest.

5 · Sign-in for the other companies — read this carefully

Three companies use this app, and **only one of them is on your tenant**:

COMPANY	ADDRESSES	HOW THEY SIGN IN
SportPharm	@sportpharm.com	Normal work login
Enovachem	whatever they use	Guest — emailed one-time code
Pharmco	whatever they use	Guest — their own login if they have M365

Pinning the app to this tenant does not lock these people out. That is the part worth being sure about before you build it, because it reads as though it would. A B2B guest is an object *inside* the SportPharm directory — Entra issues their token from your tenant, so `openIdIssuer` pinned to your tenant is exactly right and does not need changing. The same is true of "Accounts in this organizational directory only" in step 1: guests in the directory are covered by it.

What to enable:

- 1. External Identities → External collaboration settings
- 2. Guest invite settings must allow invitations. If this is set to "No one in the organization can invite guests", nothing below works.
- 3. External Identities → All identity providers → **Email one-time passcode**

Enovachem has **no Microsoft tenant** — `login.microsoftonline.com` returns `AADSTS90002: Tenant not found for enovachem.com` — so the emailed code is their route. It needs nothing from their side: no tenant, no admin, no software, no licence. Guests are free at this scale.

Inviting someone: Entra ID → Users → New user → Invite external user. Their address, and nothing else required.

Do not make the app multi-tenant to solve this. It is the obvious-looking fix and it lets any Microsoft tenant on earth sign in unless the `tid` claim is also validated against an allowlist in every Function. Guests into one pinned tenant is both simpler and tighter.

A note on Pharmco: `pharmco.com` resolves to a Microsoft tenant that is *not* SportPharm's. If that tenant is genuinely Pharmco's, their people sign in with their own work credentials as guests and nothing extra is needed. If Pharmco uses a different domain, confirm which one before inviting anyone.

What the app does with this: signing in proves who someone is. What they can see is set separately, inside HQ, on its Team page — each person is assigned to a company there. A guest who has not been assigned one sees nothing at all, which is the safe direction to fail. Partner staff only ever see the orders addressed to their own company, and never drafts.

6 • Custom domain

Use a **subdomain**, not the apex. The apex is the live store and must not move, and apex domains on Static Web Apps need ALIAS/ANAME support the DNS host may not have.

DNS for `sportpharm.com` is on `ns1/ns2.v2640474.hostpapavps.net`. `hq.sportpharm.com` is currently unused.

1. Static Web App → Custom domains → Add → `hq.sportpharm.com`
2. Add the validation `TXT` record Azure asks for
3. Add a `CNAME` for `hq` → the app's `*.azurestaticapps.net` hostname

TLS is issued and renewed by Azure. Nothing to buy, nothing to diarise.

PART 2 — THE STORE AND MAIL

Everything below is optional and can happen weeks later. HQ works without it; the pages that need it say what is missing rather than appearing broken.

None of this changes `sportpharm.com`. The WooCommerce step generates an API key from the store's admin screen — it does not install anything, alter the site, or require downtime.

P2.1 · Mail.Send, and restricting it

This is what lets HQ send orders to the vendor as `orders@sportpharm.com` instead of opening Outlook for someone to press send.

App registration → API permissions → Microsoft Graph → **Application permissions** → `Mail.Send` → **Grant admin consent**.

Then restrict it. By default `Mail.Send` as an application permission lets the app send as **any mailbox in the tenant**:

```
Connect-ExchangeOnline
New-ApplicationAccessPolicy -AppId <CLIENT_ID> `
  -PolicyScopeGroupId orders@sportpharm.com `
  -AccessRight RestrictAccess `
  -Description "SportPharm HQ may only send as the orders mailbox"
```

Verify with `Test-ApplicationAccessPolicy -Identity someone.else@sportpharm.com -AppId <CLIENT_ID>` — it should come back **denied**.

P2.2 · The remaining app settings

NAME	VALUE
TENANT_ID	Directory (tenant) ID
CLIENT_ID	Same as AAD_CLIENT_ID
CLIENT_SECRET	Same as AAD_CLIENT_SECRET
SEND_AS	orders@sportpharm.com
ALLOWED_TO	orders@sportpharm.com, AdminUnit@sportpharm.com
WOO_URL	https://sportpharm.com
WOO_KEY	WooCommerce consumer key
WOO_SECRET	WooCommerce consumer secret
STRIPE_KEY	Stripe restricted key

ALLOWED_TO **is not optional**. Without it, anyone who can sign in to HQ can use the company tenant to email anyone. It is the difference between a send button and an open relay.

P2.3 · WooCommerce key — *store admin*

WordPress → WooCommerce → Settings → Advanced → REST API → Add key.

- Description: SportPharm HQ
- Permissions: **Read** — read-only, deliberately. HQ only ever reads.

P2.4 · Stripe key — **restricted, not the secret key**

Stripe → Developers → API keys → **Create restricted key**.

Name it SportPharm HQ and grant **Read** on these, nothing else:

- Charges

- PaymentIntents
- Balance transactions

Leave every other resource at **None**. Do **not** use the standard secret key (`sk_live_...`): it can issue refunds and move money, and HQ never needs to do either. The Function checks for this and refuses to start if it is given one, rather than quietly working.

Stripe answers the half WooCommerce cannot — whether a payment actually cleared, the processing fee, refunds, disputes, and payments that carry no order reference at all (a payment link or a phone order, which will never appear in WooCommerce). The two are joined on the WooCommerce order number, which the Stripe plugin writes into each payment.

When Part 1 is done

Send Kennady:

- The Static Web App's URL (and the custom domain, if you did step 6)
- Confirmation that step 5 is done — guest invitations permitted, email one-time passcode enabled
- Nothing else — **no secrets**

That is HQ live with real logins. Part 2 can follow whenever there is time.

Afterwards

Once Part 1 is live, the **public GitHub Pages copy of HQ** should be taken down and its repository set to private. That is the temporary public host HQ runs on today — not sportpharm.com, which is untouched throughout.

Turning off Pages closes that website only; the repository history stays readable while the repository is public, so the private setting is the one that matters.

Questions you may reasonably have

Why not App Service or a VM? Nothing here needs one. It is static files plus two small HTTP functions, and Static Web Apps gives Entra auth, managed TLS and CI from the repo without configuration.

Why Entra rather than a password? So offboarding someone removes their HQ access at the same moment it removes their email, and so nobody manages another password list.

Can I see the code first? Yes — `azure/api/sendOrder/index.js` sends the order via Graph, `azure/api/woo/index.js` reads store totals and orders. Both are short and commented, and neither holds a credential; they read from the app settings above.